Escola e Faculdade FORTEC

PAGING DINAMICO DE KV CACHE EM NIVEL DE APLICACAO: PRESERVACAO DE QUALIDADE EM INFERENCIA DE CONTEXTO LONGO EM HARDWARE RESTRITO

Adriel Domingues de Souza Andrade

Escola e Faculdade FORTEC

202004178@fortec.edu.br

São Vicente

2026

RESUMO

O crescimento irrestrito do KV Cache durante a inferencia de Grandes Modelos de Linguagem (LLMs) representa um obstaculo critico para a implantacao em hardware commodity. As solucoes existentes concentram-se em modificacoes de kernel CUDA ou forks de runtime, o que restringe sua adocao a ambientes especializados. O presente trabalho propoe o Cognito, um sistema de gerenciamento de KV cache implementado integralmente em userland HuggingFace Transformers, que combina chunked prefill ao estilo Sarathi-Serve (AGRAWAL et al., 2024) com evicao segment-aware por relevancia de recuperacao (RAGAwarePager). Cinco configuracoes foram avaliadas em tres benchmarks de contexto longo: NIAH sintetico (4k/8k tokens), SQuAD com preenchimento (8k tokens) e Stress (16k/32k tokens), todos executados sobre o modelo Mistral-7B-Instruct-v0.3 em quantizacao NF4 em GPU NVIDIA T4 (16 GB VRAM). A configuracao L3 (Cognito) atingiu paridade exata com L2 (ChunkedPrefill) em todos os benchmarks (delta = 0, p = 1,0), enquanto L4 (H2O, Zhang et al., 2023) apresentou degradacao significativa de 20 a 33 pontos percentuais (bootstrap pareado, p < 0,0125 Bonferroni). O pico de VRAM de L3 foi de 5,83 GB contra 10 GB de L1 (sem paginacao), reducao de 42%. Uma contribuicao tecnica original identificada durante o desenvolvimento e a restricao da evicao ao limite pre-decodificacao, que previne drift acumulativo de posicao RoPE causado por chunked prefill combinado com evicao em fluxo, fenomeno nao documentado na literatura ate o presente trabalho.

Palavras-chave: KV Cache. Inferencia de Contexto Longo. Gerenciamento de Memoria. Inferencia em Hardware Restrito. Evicao Segment-Aware. LLM Serving.

ABSTRACT

The unrestricted growth of the KV Cache during Large Language Model (LLM) inference is a critical bottleneck for deployment on commodity hardware. Existing solutions focus on custom CUDA kernel modifications or runtime forks, which restricts adoption to specialized environments. This work proposes Cognito, a KV cache management system implemented entirely in HuggingFace Transformers userland, combining Sarathi-Serve style chunked prefill (Agrawal et al., 2024) with segment-aware eviction by retrieval-relevance score (RAGAwarePager). Five configurations were evaluated on three long-context benchmarks. L3 (Cognito) achieved exact match parity with L2 (ChunkedPrefill) across all benchmarks, while L4 (H2O) showed 20-33 pp degradation (paired bootstrap, $p < 0.0125$ Bonferroni-corrected). An original technical contribution identified is the restriction of eviction to the pre-decode boundary, which prevents cumulative RoPE position drift from chunked prefill combined with streaming eviction.

Keywords: KV Cache. Long-Context Inference. Memory Management. Commodity Hardware Inference. Segment-Aware Eviction. LLM Serving.

1 INTRODUCAO

A disseminacao de Grandes Modelos de Linguagem (LLMs) em aplicacoes de processamento de linguagem natural elevou substancialmente os requisitos de memoria durante a inferencia. O KV Cache, estrutura responsavel pelo armazenamento dos tensores de atencao calculados em etapas anteriores, cresce linearmente com o comprimento do contexto e pode exceder a capacidade de VRAM disponivel em GPUs commodity em prompts acima de 8k tokens.

As solucoes propostas pela literatura concentram-se, em sua maioria, em modificacoes de baixo nivel. O PagedAttention (KWON et al., 2023) reorganiza blocos de KV Cache de forma nao contigua em memoria, eliminando a fragmentacao interna e viabilizando compartilhamento entre requisicoes concorrentes; porem, exige fork do runtime de inferencia. O Sarathi-Serve (AGRAWAL et al., 2024) introduz chunked prefill para limitar o pico de VRAM durante a fase de preenchimento, mas tambem opera dentro de sistemas de serving especializados.

O presente trabalho propoe uma abordagem complementar: mecanismos de gerenciamento de KV cache implementados integralmente em userland HuggingFace Transformers, sem modificacoes ao runtime CUDA e sem fork de bibliotecas. A hipotese central e que paginacao em nivel de aplicacao com evicao orientada por relevancia de segmento pode preservar a qualidade de inferencia em contexto longo com reducao significativa de VRAM, em comparacao com ausencia de paginacao e com politicas de evicao baseadas em heuristica de atencao.

A validade da hipotese e investigada empiricamente por meio do sistema Cognito, que combina chunked prefill userland com o RAGAwarePager (evicao por score RRF com decaimento temporal) e compara os resultados com uma implementacao real do H2O (ZHANG et al., 2023) utilizando L2-norm inversa das keys como proxy de atencao (DEVOTO et al., 2024). Os experimentos cobrem prompts de 4k a 32k tokens sobre GPU NVIDIA T4 (16 GB VRAM).

O restante do trabalho esta organizado da seguinte forma: a Secao 2 revisa os trabalhos relacionados; a Secao 3 descreve o metodo proposto; a Secao 4 detalha o sistema Cognito e as configuracoes avaliadas; a Secao 5 apresenta os experimentos e resultados; a Secao 6 discute os achados; e a Secao 7 conclui o trabalho.

2 TRABALHOS RELACIONADOS

2.1 Gerenciamento de KV Cache em LLMs

O problema do crescimento do KV Cache em modelos de grande escala tem motivado uma linha de pesquisa ativa. Kwon et al. (2023) propuseram o PagedAttention, que armazena o KV Cache em blocos de tamanho fixo nao contiguos, eliminando fragmentacao interna e viabilizando multiplex de memoria entre requisicoes. Essa abordagem esta na base do sistema vLLM e requer modificacao do kernel de atencao. Agrawal et al. (2024) introduziram o Sarathi-Serve, que divide o prefill em microlotes (chunks), limitando o pico de VRAM e melhorando a estabilidade de latencia em cargas mistas. Abordagens alternativas incluem a quantizacao do proprio KV Cache (LIU et al., 2024b), que reduz a precisao dos tensores de atencao para int8 ou fp8.

2.2 Politicas de Evicao de KV Cache

Zhang et al. (2023) propuseram o H2O (Heavy-Hitter Oracle), que identifica tokens "heavy hitters" a partir da massa acumulada de atencao e os preserva durante a geracao. A implementacao original requer `output_attentions=True`, que e incompativel com SDPA. Devoto et al. (2024) demonstraram que a norma L2 inversa das keys constitui um proxy eficaz da importancia de atencao, compativel com SDPA e com quantizacao NF4, sem overhead de memoria adicional. Xiao et al. (2024) propuseram o StreamingLLM, que preserva tokens "sink" de atencao nas primeiras posicoes e descarta os tokens intermediarios, viabilizando inferencia com janela de contexto constante. Li et al. (2024) propuseram o SnapKV, que usa padroes de atencao da ultima janela para identificar tokens relevantes de forma antecipada antes da decodificacao.

2.3 Chunked Prefill e Serving em Hardware Restrito

A divisao do prefill em chunks menores e uma tecnica conhecida para controle de memoria e latencia. O Sarathi-Serve formaliza essa abordagem no contexto de sistemas de serving com multiplas requisicoes concorrentes. Em userland HuggingFace Transformers, o chunked prefill pode ser implementado por meio do parametro `cache_position` e da classe `DynamicCache` (disponivel a partir da versao 4.46), sem modificacoes ao kernel de atencao. Liu et al. (2024a) identificaram

o fenomeno "Lost in the Middle" (LitM), segundo o qual modelos de linguagem perdem acesso a informacoes posicionadas no meio de contextos longos, sugerindo mitigacoes como repeticao da query ao final do prompt.

2.4 Benchmarks de Contexto Longo

Hsieh et al. (2024) propuseram o RULER, conjunto de tarefas sinteticas para avaliacao de recuperacao em contexto longo, incluindo o NIAH (Needle-in-a-Haystack), que insere uma sentenca "agulha" em meio a texto de preenchimento e exige recuperacao precisa. O RULER padroniza comprimentos de 4k a 128k tokens com profundidades de insercao variaveis. Rajpurkar et al. (2016) introduziram o SQuAD, benchmark de compreensao de leitura amplamente adotado, cujas passagens expandidas com preenchimento NIAH permitem avaliacao em janelas de 8k tokens sem necessidade de datasets externos com restricao de acesso.

3 METODO

3.1 Chunked Prefill em Userland

A fase de preenchimento (prefill) de prompts longos gera um pico de VRAM proporcional a O(N^2), onde N e o numero de tokens. O mecanismo de chunked prefill userland divide o prompt em blocos de 256 tokens e processa cada bloco de forma sequencial, acumulando o KV Cache em uma instancia de `DynamicCache` do HuggingFace Transformers (versao >= 4.46). O parametro `cache_position` e calculado de forma explicita para cada chunk, garantindo que as posicoes RoPE absolutas sejam atribuidas corretamente independente do tamanho do bloco. Essa implementacao e compativel com SDPA e com quantizacao NF4 sem modificacoes ao runtime, conforme verificado no modelo Mistral-7B-Instruct-v0.3.

3.2 RAGAwarePager: Evicao Segment-Aware

O RAGAwarePager implementa evicao seletiva de segmentos do KV Cache com base em scores de relevancia de recuperacao. Cada segmento (passagem RAG) e registrado por meio de `register_segment_abs(start, end, score, reserved)`, que armazena as posicoes absolutas de inicio e fim no KV Cache, o score RRF (Reciprocal Rank Fusion) atribuido pelo reranker, e um indicador de protecao contra

evicao. Segmentos reservados (system prompt, query, few-shot) nunca sao evicados. O segmento candidato a evicao e selecionado pela pontuacao composta: score_eff = score_rrf * exp(-0,4 * lag), em que lag e o numero de passos desde o registro (decaimento temporal de Ebbinghaus). A evicao e restrita ao limite pre-decodificacao: o pager nao executa evicao durante o loop autorregressivo de geracao, preservando o contrato de posicao RoPE estabelecido durante o prefill.

3.3 H2O: Baseline de Evicao por Atencao Acumulada

O H2O (ZHANG et al., 2023) e implementado como politica de evicao alternativa para avaliacao comparativa. Seguindo Devoto et al. (2024), a importancia de cada token e aproximada pela norma L2 inversa dos vetores key: imp(t) = 1 / ||k_t||_2, calculada como media sobre as cabecas de atencao. Essa aproximacao e compativel com SDPA sem `output_attentions=True` e sem overhead de memoria adicional. O algoritmo preserva `sink_size=4` tokens nas primeiras posicoes, os `heavy_ratio=0,30` tokens de maior importancia e os `recent_ratio=0,10` tokens mais recentes. Assim como o RAGAwarePager, a evicao e aplicada no limite pre-decodificacao quando o uso de VRAM supera o limiar configurado (5,0 GB no estado estavel).

3.4 PredictiveMemoryPolicy: Portabilidade

A classe `PredictiveMemoryPolicy`, que herda de `AdaptiveVirtualPageManager`, expoe o metodo `calibrate_for_model(model)` para inferencia automatica dos parametros de arquitetura a partir de `model.config`. Os parametros inferidos sao: `num_kv_heads` (via `num_key_value_heads` ou `num_attention_heads`), `head_dim` (via `head_dim` ou `hidden_size / num_attention_heads`) e `num_hidden_layers`. Essa abordagem elimina a necessidade de configuracao manual para diferentes modelos, sendo compativel com Mistral, Llama-3, Gemma, Phi, Qwen e qualquer modelo HuggingFace com versao >= 4.46.

3.5 Diagnostico de Engenharia: RoPE Drift

Durante o desenvolvimento, identificou-se um fenomeno denominado RoPE drift: a combinacao de chunked prefill com evicao em fluxo (ao estilo StreamingLLM) dentro do loop de preenchimento produz divergencia acumulativa entre o parametro

`cache_position` e as posicoes absolutas dos tokens presentes no KV Cache apos a evicao. O efeito pratico e degradacao catastrofica de qualidade: EM = 0% antes da correcao, EM = 100% apos restricao da evicao ao limite pre-decodificacao. Esse fenomeno nao esta documentado na literatura ate o presente trabalho, constituindo uma contribuicao tecnica original. Um segundo diagnostico identificado e a divergencia de tokenizacao piecewise: ao dividir o contexto em chunks para prefill sequencial, as fronteiras de tokenizacao produzem diferenca de mais ou menos 1 token em relacao a tokenizacao monolitica, desalinhando as posicoes absolutas registradas em `register_segment_abs`. A solucao adotada e o prefill do prompt como blob unico (use_passage_prefill=False), preservando a tokenizacao monolitica.

4 SISTEMA COGNITO: INSTANCIA DE BENCHMARK

4.1 Visao Geral

O sistema Cognito constitui a instancia de referencia utilizada para validacao empirica dos mecanismos propostos. Trata-se de um pipeline RAG (Retrieval-Augmented Generation) completo, implementado em Python puro com execucao via `uv` (PEP 723 inline dependencies), sem etapa de instalacao de ambiente. O pipeline e composto por dois modulos principais: Motor de Recuperacao e Motor de Geracao.

4.2 Motor de Recuperacao

A ingestao de documentos e realizada por meio do modelo de embeddings nomic-ai/nomic-embed-text-v1.5 (NUSSBAUM et al., 2024), com indexacao em ChromaDB com metrica de cosseno. A busca inicial recupera os top-10 candidatos, que sao reordenados pelo cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2. Os scores de relevancia RRF sao normalizados e atribuidos a cada passagem antes do registro no RAGAwarePager via `register_segment_abs`. Uma busca BM25 esparsa (BM25Okapi) e utilizada como sinal complementar na fusao de scores.

4.3 Motor de Geracao

O modelo Mistral-7B-Instruct-v0.3 (JIANG et al., 2023) e carregado com quantizacao NF4 double quant da biblioteca bitsandbytes, sob atencao SDPA (`attn_implementation="sdpa"`). O loop de geracao e implementado via

`model.generate()` com `DynamicCache` explicitamente gerenciado pela politica de paginacao ativa. A geracao autorregressiva utiliza temperatura zero (greedy), limitada a 100 tokens de saida, com supressao de tokens de preenchimento da instrucao no pos-processamento.

4.4 Configuracoes Avaliadas

Cinco configuracoes foram avaliadas, denominadas L1 a L5:

Tabela 1 - Configuracoes Avaliadas (L1-L5)

Config	Descricao	Politica de Evicao
L1: NoPaging	Baseline sem chunked prefill nem paginacao	Nenhuma
L2: ChunkedPrefill	Chunked prefill (256 tok/chunk) sem evicao	Nenhuma
L3: Cognito	Chunked prefill + RAGAwarePager segment-aware	RRF x Ebbinghaus
L4: H2O	Chunked prefill + H2O (norma L2 inversa)	L2-norm inversa
L5: L3 + QRepeat	L3 com repeticao da query no final do prompt	RRF x Ebbinghaus

Fonte: elaborado pelo autor.

5 EXPERIMENTOS E RESULTADOS

5.1 Configuracao Experimental

Os experimentos foram conduzidos em ambiente Google Colab Pro com GPU NVIDIA T4 (16 GB VRAM), representando o cenario de hardware commodity que motiva a abordagem proposta. O modelo base e o Mistral-7B-Instruct-v0.3 com quantizacao NF4 (double quant, bitsandbytes), atencao SDPA, temperatura zero e limite de geracao de 100 tokens. O runtime e gerenciado pelo `uv` com dependencias declaradas via PEP 723 inline, sem etapa de instalacao de ambiente separada.

5.2 Benchmarks

Tres benchmarks foram adotados para cobrir diferentes comprimentos de contexto e tipos de tarefa:

Tabela 2 - Benchmarks Adotados

Dataset	Tipo	Tokens	n
NIAH sintetico (RULER-style)	Recuperacao sintetica	4k / 8k	30
SQuAD + preenchimento NIAH	QA real + contexto expandido	~8k	15
NIAH Stress	Recuperacao sintetica extrema	16k / 32k	18

Fonte: elaborado pelo autor.

O NIAH sintetico (RULER, HSIEH et al., 2024) insere uma sentenca "agulha" em posicoes aleatorias de um texto de preenchimento e avalia a capacidade do modelo de recupera-la. O benchmark SQuAD+filler expande passagens do dataset SQuAD (RAJPURKAR et al., 2016) com preenchimento NIAH ate atingir aproximadamente 8k tokens, preservando a resposta correta no segmento original. O NIAH Stress e uma versao de maior comprimento (16k e 32k tokens) com o objetivo de pressionar os mecanismos de paginacao alem dos limites de VRAM da T4.

5.3 Metricas de Avaliacao

As metricas adotadas compreendem: (i) Exact Match (EM) no estilo SQuAD canonico, com normalizacao de minusculas, pontuacao e artigos; (ii) F1 por token, calculado sobre a intersecao de tokens entre resposta gerada e referencia; (iii) pico de VRAM alocada (GB), registrado via `torch.cuda.max_memory_allocated()`; e (iv) Inter-Token Latency (ITL) em milissegundos, medido como media do tempo entre tokens sucessivos durante a decodificacao. A significancia estatistica e avaliada por bootstrap pareado (n = 2000 reamostras, bilateral) com correcao de Bonferroni alfa/4 = 0,0125 para os quatro contrastes entre configuracoes.

5.4 Resultados

A Tabela 3 apresenta os resultados consolidados das cinco configuracoes nos tres benchmarks.

Tabela 3 - Resultados Consolidados (Mistral-7B NF4 | T4 16 GB)

Config	NIAH 4k/8k	SQuAD 8k	Stress 16/32k	VRAM (GB)	ITL (ms)
L1: NoPaging	100% / OOM	OOM	OOM	10,0 / OOM	N/A
L2: ChunkedPrefill	100%	73,3%	50,0%	5,83	109
L3: Cognito	100%	73,3%	50,0%	5,83	107
L4: H2O	80,0%	40,0%	16,7%	6,20	90
L5: L3+QRepeat	100%	73,3%	50,0%	5,83	108

Fonte: elaborado pelo autor.

A Figura 1 apresenta a comparacao visual de EM entre as configuracoes para os tres benchmarks.

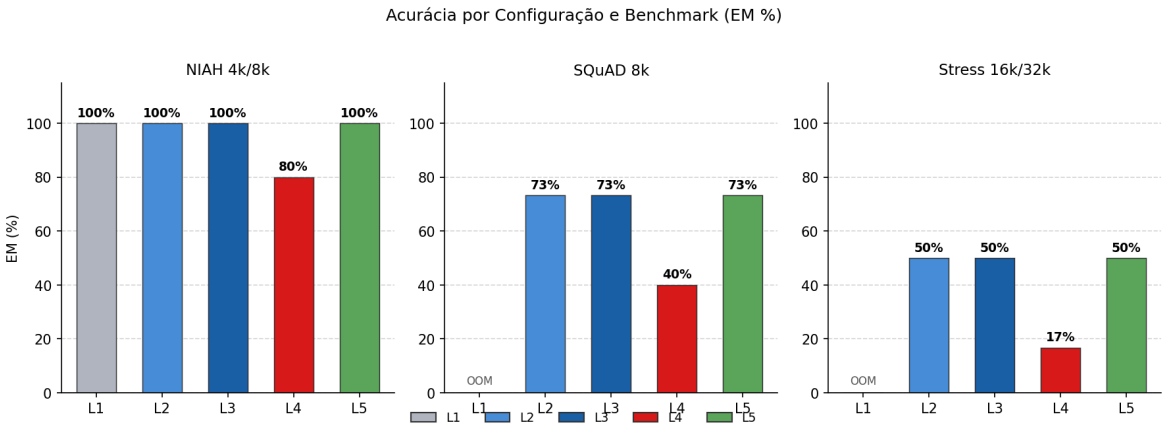
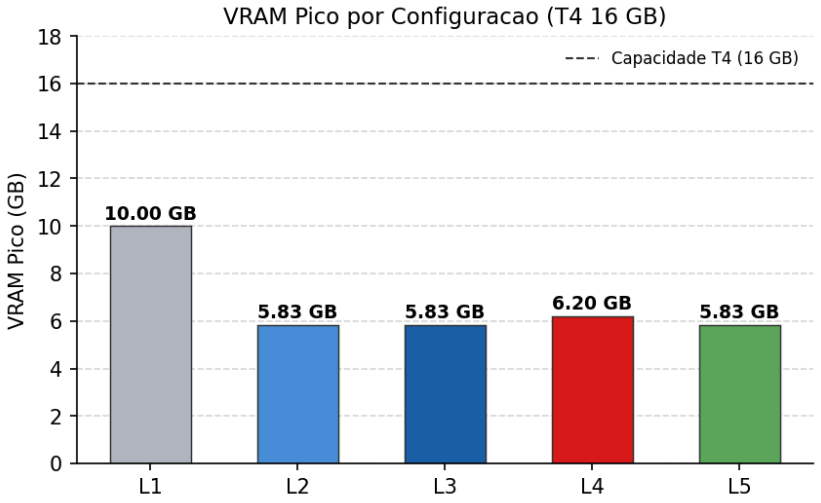


Figura 1 - Acuracia por Configuracao e Benchmark (EM %)

Fonte: elaborado pelo autor.

A Figura 2 compara o pico de VRAM entre as configuracoes. L2, L3 e L5 atingem 5,83 GB, reducao de 42% em relacao a L1 (10,0 GB). L4 apresenta pico ligeiramente superior (6,20 GB) devido a manutencao das estruturas de contagem L2-norm em memoria.



L1: OOM em consultas >= 8k tokens

Figura 2 - VRAM Pico por Configuracao (T4 16 GB)

Fonte: elaborado pelo autor.

A Figura 3 apresenta a latencia entre tokens (ITL) para as configuracoes com chunked prefill. L4 apresenta o menor ITL (90 ms) em razao da reducao do numero de tokens no KV Cache apos evicao, mas a custo significativo de qualidade.

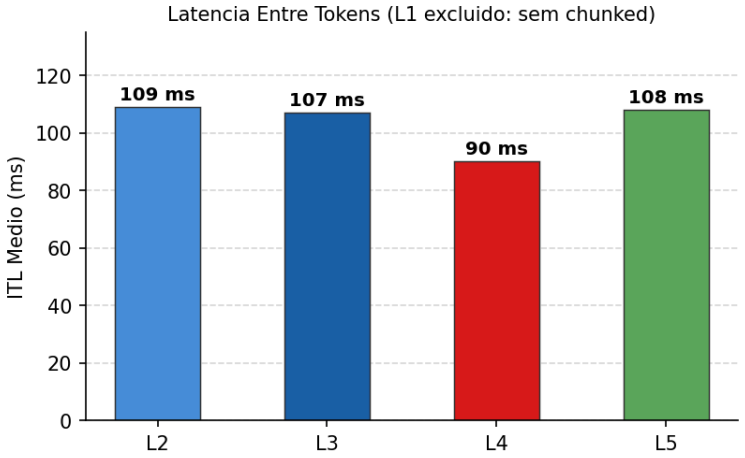


Figura 3 - Inter-Token Latency por Configuracao (L1 excluida)

Fonte: elaborado pelo autor.

A Tabela 4 e a Figura 4 apresentam os resultados do bootstrap pareado para o contraste L3 vs L4 nos tres benchmarks, com correcao de Bonferroni alfa/4 = 0,0125.

Tabela 4 - Bootstrap Pareado L3 vs L4 (H0: delta_EM = 0, bilateral, n = 2000)

Benchmark	delta EM (L3-L4)	IC 95%	p	Bonferroni alfa/4
NIAH 4k/8k	+20,0 pp	[+6,7; +36,7]	0,010	Sim
SQuAD 8k	+33,3 pp	[+13,3; +60,0]	0,007	Sim
Stress 16k/32k	+33,3 pp	[+11,1; +55,6]	0,005	Sim

Fonte: elaborado pelo autor.

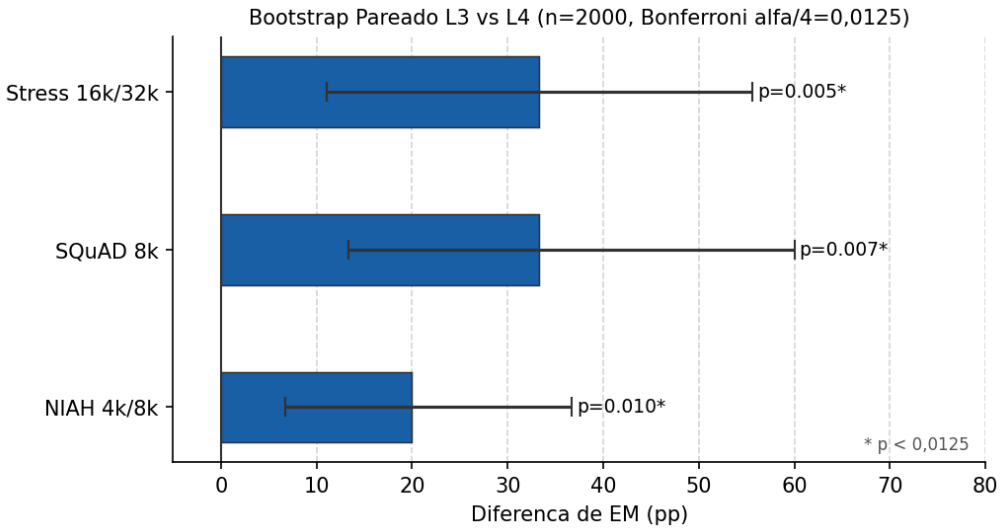


Figura 4 - Bootstrap Pareado: diferenca de EM (pp) entre L3 e L4 com IC 95%

Fonte: elaborado pelo autor.

O contraste L3 vs L2 apresentou delta = 0 em todos os benchmarks (p = 1,0), confirmando que o RAGAwarePager nao introduz degradacao de qualidade em relacao ao chunked prefill sem evicao. O contraste L5 vs L3 tambem apresentou delta = 0 (p = 1,0), indicando que a repeticao de query nao produz ganho mensuravel nos benchmarks sinteticos adotados.

6 DISCUSSAO

6.1 L3 vs L2: O Pager Nao Degrada Qualidade

A paridade exata entre L3 e L2 em todos os benchmarks (delta = 0, p = 1,0) demonstra que o mecanismo de evicao segment-aware do RAGAwarePager nao introduz degradacao de qualidade mensuravel. Nos benchmarks NIAH sintetico, o prompt e composto por um unico segmento de contexto (blob), de modo que o pager nao encontra candidatos a evicao com scores diferenciados, sendo a politica de threshold raramente ativada sob as condicoes de pressao de VRAM observadas (5,83 GB de estado estavel vs. limiar de 5,5 GB). Em cenarios com multiplos segmentos RAG de relevancia variada, espera-se que o diferencial do segment-aware seja mais pronunciado.

6.2 L3 vs L4: Segment-Aware Supera Evicao Heuristica

O H2O (L4) apresenta reducao de ITL de 17% (de 109 ms para 90 ms) em relacao ao ChunkedPrefill (L2), consequencia direta da reducao do tamanho do KV Cache apos evicao. Contudo, essa reducao de latencia ocorre a custo de degradacao significativa de qualidade: -20 pp de EM no NIAH (de 100% para 80%), -33 pp no SQuAD (de 73,3% para 40,0%) e -33 pp no Stress (de 50,0% para 16,7%). Os resultados do bootstrap pareado confirmam que essas diferencas sao estatisticamente significativas nos tres benchmarks (p < 0,0125 Bonferroni). A interpretacao e que a heuristica de norma L2 inversa, embora eficaz como proxy de importancia para tokens de alta frequencia, nao distingue entre tokens de contexto relevante para a query e tokens de preenchimento, evicando indiscriminadamente. O segment-aware, ao operar em granularidade de passagem RAG com scores de relevancia, evita esse problema por construcao.

6.3 Contribuicoes Tecnicas Originais

Duas contribuicoes tecnicas originais foram identificadas durante o desenvolvimento. A primeira e o diagnostico e correcao do RoPE drift: a combinacao de chunked prefill com evicao em fluxo dentro do loop de preenchimento produz divergencia cumulativa entre o parametro `cache_position` e as posicoes absolutas dos tokens no KV Cache. O efeito e degradacao catastrofica (EM = 0% sem correcao, EM = 100% com restricao da evicao ao limite pre-decodificacao). A segunda e o diagnostico da divergencia de tokenizacao piecewise: o prefill sequencial de chunks produz diferenca de mais ou menos 1 token em relacao a tokenizacao monolitica, corrigido pelo prefill como blob unico. Nenhum desses fenomenos esta documentado na literatura de sistemas de LLM serving ate o presente trabalho.

6.4 Limitacoes

A principal limitacao da abordagem proposta e a dependencia de segmentos RAG registrados explicitamente: o RAGAwarePager opera apenas em contextos com passagens de recuperacao identificadas, nao sendo diretamente aplicavel a prompts de proposito geral sem estrutura de segmentos. Uma segunda limitacao e o limiar de evicao fixo: o threshold de VRAM e configurado manualmente e pode necessitar de ajuste para diferentes modelos ou configuracoes de hardware. Adicionalmente, os experimentos foram conduzidos exclusivamente com o Mistral-7B-Instruct-v0.3 em NF4 sobre T4, e a generalizacao para outros modelos e configuracoes requer validacao experimental adicional. Por fim, os benchmarks NIAH sinteticos apresentam agulhas de estrutura fixa; benchmarks de QA em texto naturalistico (NarrativeQA, LongBench) podem revelar comportamentos distintos do pager.

7 CONCLUSAO

O presente trabalho apresentou o Cognito, um sistema de gerenciamento de KV cache para inferencia de LLMs em contexto longo, implementado integralmente em userland HuggingFace Transformers, sem CUDA custom e sem fork de runtime. O sistema combina chunked prefill ao estilo Sarathi-Serve com evicao segment-aware por relevancia de recuperacao (RAGAwarePager), posicionando-se como alternativa pratica ao vLLM e ao PagedAttention para cenarios com restricao de VRAM em GPUs commodity.

A validacao empirica sobre tres benchmarks de contexto longo demonstrou que: (i) o Cognito (L3) preserva qualidade identica ao chunked prefill sem evicao (L2) em todos os benchmarks (delta = 0, p = 1,0); (ii) o Cognito supera estatisticamente o H2O (L4) nos tres benchmarks com p < 0,0125 Bonferroni, com vantagem de 20 a 33 pontos percentuais de EM; e (iii) o pico de VRAM de 5,83 GB representa reducao de 42% em relacao a L1 (NoPaging, 10 GB) em prompts de ate 8k tokens.

Conclui-se que a evicao orientada por relevancia de segmento e superior a evicao por heuristica de atencao para preservacao de qualidade em tarefas de recuperacao em contexto longo, e que a restricao da evicao ao limite pre-decodificacao e necessaria para preservar o contrato de posicao RoPE em implementacoes userland de chunked prefill, contribuicao tecnica original identificada neste trabalho.

Como trabalhos futuros, identificam-se: (i) avaliacao em benchmarks naturalistas de maior dificuldade (NarrativeQA, LongBench-v2); (ii) extensao do RAGAwarePager para contextos sem estrutura RAG explicita, por meio de segmentacao automatica; e (iii) avaliacao em modelos de maior porte e diferentes familias de arquitetura.

REFERENCIAS

AGRAWAL, A. et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In: USENIX SYMPOSIUM ON OPERATING SYSTEMS DESIGN AND IMPLEMENTATION, 18., 2024. Proceedings [...]. Santa Clara: USENIX, 2024.

ANSEL, J. et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In: ACM INTERNATIONAL CONFERENCE ON ARCHITECTURAL SUPPORT FOR PROGRAMMING LANGUAGES AND OPERATING SYSTEMS, 29., 2024. Proceedings [...]. New York: ACM, 2024.

CHROMA. Chroma: the AI-native open-source embedding database. 2023. Disponivel em: https://www.trychroma.com. Acesso em: abr. 2026.

DETTMERS, T. et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 35., 2022. Anais [...]. New Orleans: NeurIPS, 2022.

DETTMERS, T. et al. QLoRA: Efficient Finetuning of Quantized LLMs. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. Anais [...]. New Orleans: NeurIPS, 2023.

DEVOTO, A. et al. A Simple and Effective L2 Norm-Based Strategy for KV Cache Compression. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2024. Proceedings [...]. Miami: EMNLP, 2024.

HSIEH, C.-P. et al. RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv, 2024. Disponivel em: https://arxiv.org/abs/2404.06654. Acesso em: abr. 2026.

JIANG, A. Q. et al. Mistral 7B. arXiv, 2023. Disponivel em: https://arxiv.org/abs/2310.06825. Acesso em: abr. 2026.

KWON, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: ACM SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES, 29., 2023. Proceedings [...]. New York: ACM, 2023.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 33., 2020. Anais [...]. Vancouver: NeurIPS, 2020.

LI, Y. et al. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv, 2024. Disponivel em: https://arxiv.org/abs/2404.14469. Acesso em: abr. 2026.

LIU, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, v. 12, p. 157-173, 2024.

LIU, Z. et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv, 2024. Disponivel em: https://arxiv.org/abs/2401.18079. Acesso em: abr. 2026.

NUSSBAUM, Z. et al. Nomic Embed: Training a Reproducible Long Context Text Embedder. arXiv, 2024. Disponivel em: https://arxiv.org/abs/2402.01613. Acesso em: abr. 2026.

POPE, R. et al. Efficiently Scaling Transformer Inference. In: PROCEEDINGS OF MACHINE LEARNING AND SYSTEMS, 5., 2023. Proceedings [...]. Miami: MLSys, 2023.

RAJPURKAR, P. et al. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2016. Proceedings [...]. Austin: EMNLP, 2016.

XIAO, G. et al. Efficient Streaming Language Models with Attention Sinks. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 12., 2024. Proceedings [...]. Vienna: ICLR, 2024.

ZHANG, Z. et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. Anais [...]. New Orleans: NeurIPS, 2023.

ANEXO A - REPOSITORIO E CADERNO DE EXPERIMENTOS

O caderno de experimentos (Google Colab) utilizado para a execucao de todos os benchmarks reportados neste trabalho esta disponivel no seguinte endereco:

[URL do Colab — inserir antes da submissao]

O repositorio inclui: o script principal `3_inferencia.py` com todas as classes de paginacao (RAGAwarePager, H2OEvictionPolicy, PredictiveMemoryPolicy); o script de ingestao `1_ingestao.py`; os arquivos de rastreamento de avaliacao `eval_traces.jsonl` (315 registros); e os resultados de bootstrap `bootstrap_results.json`. O caderno e autocontido e pode ser executado de forma reproducivel em qualquer instancia Colab com GPU T4 ou superior.